

Singleton Pattern: only 1 database is created throughout the application

```
0 references
internal class Database
{
    private static Database _instance;
    private static readonly object _lock = new object();

    0 references
    public List<Document> Documents { get; set; } = new List<Document>();
    0 references
    public List<User> Users { get; set; } = new List<User>();
    0 references
    public List<Loan> Loans { get; set; } = new List<Loan>();

    1 reference
    private Database()
    {
    }

    1 reference
    public static Database Instance
    {
        get
        {
            lock (_lock)
            {
                if (_instance == null)
                {
                    _instance = new Database();
                }
                return _instance;
            }
        }
    }
}
```

Factory Method Pattern: involves creating an abstract class Document, and then defining concrete classes like Book, Magazine, and Newspaper, each implementing the abstract method(s) from Document

```
3 references
internal abstract class Document
{
    3 references
    public string Title { get; set; }
    3 references
    public abstract void DisplayInfo();
}
```

0 references

```
internal class Book : Document
{
    1 reference
    public override void DisplayInfo()
    {
        Console.WriteLine($"Book: {Title}");
    }
}
```

```
internal class Magazine : Document
{
    public override void DisplayInfo()
    {
        Console.WriteLine($"Magazine: {Title}");
    }
}
```

0 references

```
internal class Newspaper : Document
{
    1 reference
    public override void DisplayInfo()
    {
        Console.WriteLine($"Newspaper: {Title}");
    }
}
```

Observer Pattern: involves creating a notification system for each library that is created. Whenever a new library is added, all registered observers should be notified

5 references

```
internal interface INotification
{
    2 references
    void Notify(string message);
}
```

```

0 references
internal class Library
{
    List<INotification> notifications = new List<INotification>();
    0 references
    public void AddObserver(INotification notification) => notifications.Add(notification);
    0 references
    public void RemoveObserver(INotification notification) => notifications.Remove(notification);
    0 references
    public void Notify(string message)
    {
        foreach (var notification in notifications)
        {
            notification.Notify(message);
        }
    }
}

```

Strategy Pattern: involves defining an interface that shares a function with all its implementing classes. A separate class uses this interface to access the function, which was declared earlier. This allows dynamic selection of different behaviors at runtime.

```

3 references
internal interface ILoanFeeCalculation
{
    4 references
    decimal CalculateFee(int days);
}

```

```

0 references
internal class BookLoanFeeCalculation : ILoanFeeCalculation
{
    2 references
    public decimal CalculateFee(int days)
    {
        return days * 100m;
    }
}

```

```

0 references
internal class Loan
{
    private ILoanFeeCalculation _loanFeeCalculation;
    0 references
    public Loan(ILoanFeeCalculation loanFeeCalculation)
    {
        _loanFeeCalculation = loanFeeCalculation;
    }
    0 references
    public decimal GetLoanFee(int days) => _loanFeeCalculation.CalculateFee(days);
}

```